AI Audit Logging Architecture

Enterprise Architecture · AI Strategy & Standards · Oct 11, 2026 · @Evan Ovalle

Executive summary

Every AI interaction that touches firm data on behalf of a third party must produce a tamper-evident, replayable audit trail that links the consumer's request, every tool and data call the agent made, every value it saw, and the answer it returned, held for the same retention we apply to books and records. Today the firm cannot answer "what did the AI tell this advisor, from which data, and who authorised it" for most pilots. This document sets the standard that closes that gap and the platform that makes it the default rather than a per-team burden.

The decision is to treat AI audit logging as a shared control plane, the **AI Audit Plane**, not as application logging. Five commitments bind every team exposing AI to a third party:

1. **Log at the chokepoints, not inside the model.** Four mandatory enforcement points capture everything: the Consumer Edge Gateway, the Agent Runtime, the Model Gateway and the Tool/MCP Gateway. Teams build on them; they do not re-implement them.
2. **One canonical record.** Every event is an `AIAuditEvent` in a fixed schema with a shared correlation spine (interaction, trace, span, step), a verified identity chain (end investor, advisor, advisory firm, agent, service) and content-addressed payloads.
3. **Provenance is a first-class field.** Every retrieved value and every tool result carries a source reference and content hash. Every model output is linked to the exact retrieval and tool spans that grounded it, so a claim can be traced to the record it came from.
4. **Tamper evidence by construction.** Events are hash-chained per stream, Merkle-batched, signed by the platform and anchored to an external timestamp authority. Primary storage is write-once (WORM) and retention-locked.
5. **Retention follows the books-and-records rule.** Decision-bearing AI interactions with third parties are retained 7 years in WORM (SEC 17a-4 / FINRA 4511 posture), with privacy-tiered redaction and an index that keeps them searchable within 24 hours of a request.

The platform ships as a paved road: a gateway, a language SDK, a sidecar for teams that cannot change code, and a conformance test that certifies a workload before it is exposed externally.

Scope

The standard applies to any production path where a model, agent or AI-mediated tool acts on firm data for a party outside the firm, and to the internal AI services those paths depend on. It applies regardless of hosting model (our cloud, a vendor API, an on-premises model) and regardless of who authored the agent.

Exposure surface	What the third party gets	Who drives the model	Audit emphasis
MCP servers we publish	Tools and resources over the Model Context Protocol, called by the advisory firm's own assistant or agent	The third party	We only see tool calls, not their prompts. Log the full tool request, result, identity and data lineage. Treat every call as an unattended API call with an untrusted caller.
Hosted AI assistants (chat, embedded widgets)	A conversational interface backed by our models and data	The firm	Full prompt, retrieval, tool and response capture. Disclosures and disclaimers shown must be in the record.
Backend agents run on request	An autonomous multi-step workflow (rebalance proposal, document pack, account review) executed by our runtime on the third party's trigger	The firm	Every step, plan revision, tool call and intermediate value. Approval and human-in-the-loop events.
Third-party agents calling our APIs with AI-generated requests	Conventional REST/GraphQL APIs, but the caller is an agent	The third party	Agent attestation headers, intent capture, anomaly signals. Existing API logs extended with the AI envelope.

Exposure surface	What the third party gets	Who drives the model	Audit emphasis
Batch and event-driven AI (nightly document extraction, alerts to advisors)	Outputs delivered later, not interactively	The firm	Same record, correlated by job rather than session.
Internal AI services consumed by the above (embedding, classification, summarisation)	Nothing directly	The firm	Logged as child spans so the external record is complete.

Out of scope for this document: model training data governance, model evaluation and red-teaming records, and employee-facing productivity assistants that never touch client data. Those have their own standards and share the storage platform described here.

Terms used throughout: **consumer** is the advisory firm or its system; **principal** is the human or account the action is ultimately for; **interaction** is one end-to-end request from a consumer to a final answer or completed workflow; **step** is one model call or tool call inside it.

Risk model

AI between a third party and firm data changes the risk profile in three ways: the caller's intent is expressed in natural language and translated by a model we do not fully control, the model can combine data from many sources into a single plausible answer, and the model's output can drive actions. Each risk below names the log evidence that lets us detect, investigate and defend.

Risk	How it shows up	What the log must prove
Hallucinated or misattributed facts	The assistant states a holding, fee or performance figure that is not in any record, or attributes it to the wrong account	Every number and claim in the response links to a retrieval or tool span with a content hash of the source value. Absence of a link is itself evidence.

Risk	How it shows up	What the log must prove
Over-retrieval and data leakage	The agent pulls more accounts, documents or fields than the request needed, or the consumer is shown data for a principal they do not represent	Full tool request parameters, the authorisation decision for each call, the row and field counts returned, the principal scope asserted versus the scope actually touched.
Prompt injection via third-party content	A document, email or web page the agent reads contains instructions that change its behaviour	Hash and source of every piece of content entering the context window, the model's tool-call plan before and after that content arrived, and the injection classifier verdict.
Confused deputy through MCP	The consumer's own agent chains our tools in an order we did not anticipate, escalating what it can see or do	Per-call identity chain (firm, agent, principal), the consumer's session id, our tool's own policy decision, and rate and sequence anomalies across calls.
Unauthorised or unintended actions	The agent submits an order, changes a beneficiary or sends a document without a valid human approval	Action-class tagging on every tool, the approval event with approver identity and what they were shown, the dry-run or plan the model produced before acting.
Model and prompt drift	A model version or system prompt change alters answers to the same question	Model id, version, provider, system prompt hash, tool manifest hash, temperature and all decoding parameters on every model span.
Vendor-side processing	A hosted model provider retains, trains on, or is breached with our prompts	Provider, region, data-handling contract id and zero-retention flag recorded per call; our own copy of the prompt and response so we never depend on the vendor's logs.

Risk	How it shows up	What the log must prove
Non-repudiation disputes	The advisory firm claims the assistant said something it did not, or the firm claims a request was never made	Signed, timestamped, hash-chained records on both request and response, with the exact rendering shown to the user where we control the UI.
Regulatory inquiry	Examiner asks for every AI-assisted communication with a given firm over a period	Search by consumer, principal, time, model, tool and outcome within 24 hours, exported with integrity proofs.
Log tampering or loss	An insider or attacker alters or deletes evidence	Append-only streams, external anchoring, segregation of duties between producers, custodians and auditors, and continuous chain verification.

The common thread is that conventional request logging captures the envelope. AI logging must capture the **content**, the **context the model saw**, the **decisions made along the way**, and the **lineage** from source to statement. That is why the enforcement points in the next sections sit on the data path rather than beside it.

Design principles

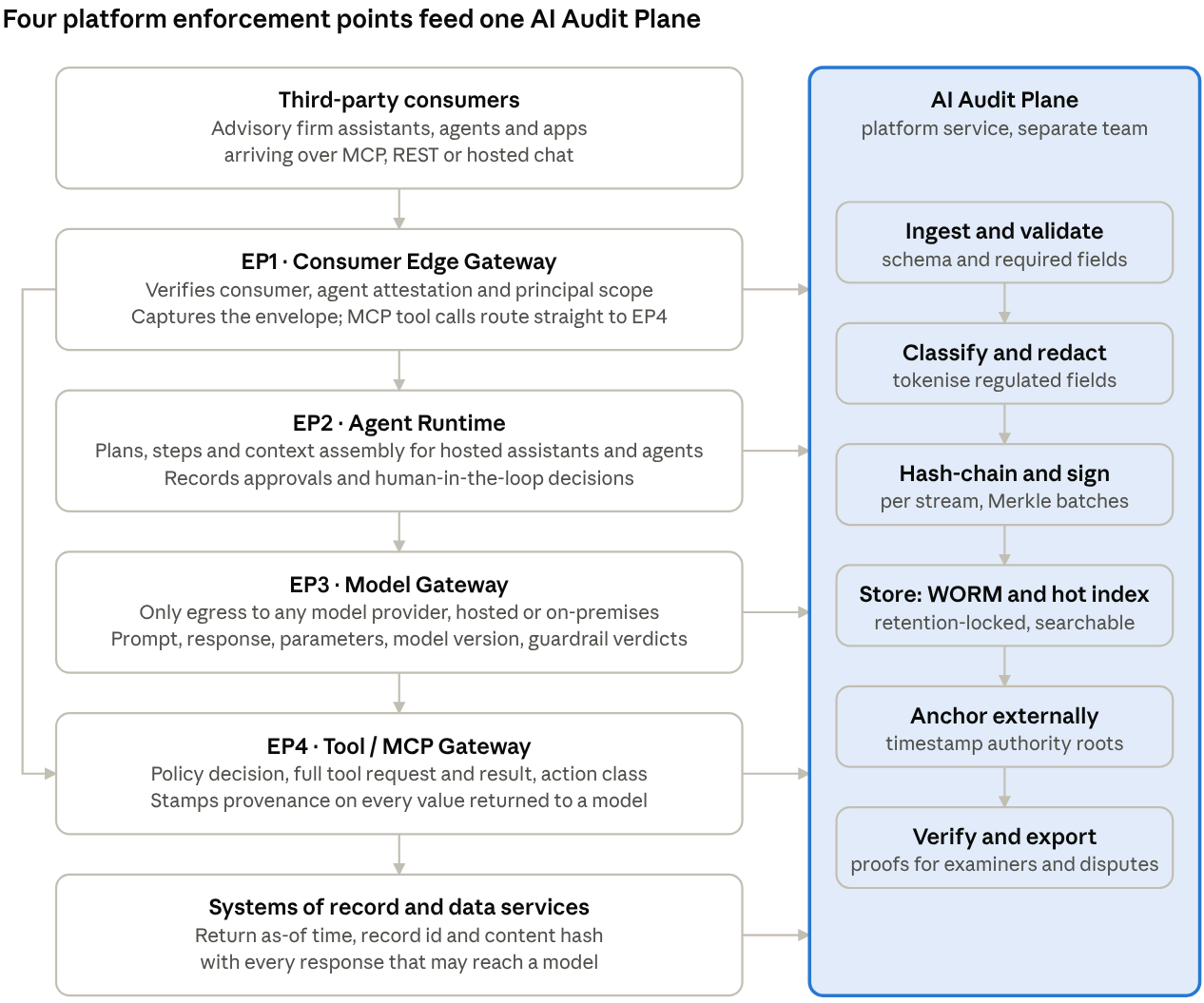
These ten principles are the test every design review applies. A workload that cannot satisfy one of them is not exposed to a third party.

1. **Capture at the chokepoint, never trust the application to remember.** Logging lives in platform-owned gateways and runtimes that sit on the data path. Application code may enrich a record; it cannot be the only thing that creates one.
2. **Fail closed for high-risk actions, fail open with alarm for read-only answers.** If the audit pipeline cannot accept a record for a transactional tool call, the call does not proceed. For read-only retrieval, the call proceeds, the record is buffered locally, and a platform alert fires.

3. **Log the content, reference the bulk.** Prompts, responses and tool payloads are stored in full. Large artefacts (documents, result sets over a threshold) are stored once in content-addressed blob storage and referenced by hash, so the record stays small and the artefact stays verifiable.
4. **Every value has a source.** Any datum entering a model context carries a provenance stamp: system of record, query or tool, record identifier, as-of time, content hash. Any claim leaving the model is linked back to those stamps.
5. **Identity is a chain, not a field.** Records carry the consumer organisation, the calling system or agent, the human operator where present, the principal the request is for, and the firm service acting. Each link is verified, not asserted.
6. **Immutable at write, provable at read.** Append-only streams, hash chaining, platform signatures and external anchoring. Any exported record can be verified offline against a published root.
7. **Minimise what we keep, but never minimise what we need.** Redaction follows a data classification, is applied by the platform at ingest using reversible tokenisation for regulated fields, and the raw form is retained only in the restricted vault tier. Nothing in the decision path is dropped.
8. **Deterministic replay.** With the record alone, an investigator can reconstruct the exact context window, tool manifest, parameters and model version and re-run the step against a frozen model to compare outputs.
9. **One schema, many sinks.** Teams emit `AIAuditEvent` through the SDK or gateway; the platform routes to hot search, WORM archive, SIEM and analytics. No team runs its own audit store.
10. **Observable by default, exportable by request.** Operational telemetry (OpenTelemetry traces and metrics) and the audit record share identifiers but are different systems with different retention and access. The audit record is the legal artefact; traces are for engineers.

Reference architecture

Four platform-owned enforcement points sit on the data path between a third party and firm data, and every one of them emits into a single AI Audit Plane. Application teams integrate with the enforcement points; they never write to the audit store directly.



AI audit reference architecture · 4 enforcement points, 1 audit plane

Traffic flows down the left column; every enforcement point and every system of record emits into the Audit Plane on the right. A third-party MCP client goes from the edge gateway straight to the tool gateway, so the loop on the left is the only path that skips the runtime and model gateway.

Enforcement point	Where it sits	What it owns	Mandatory for	How a team adopts it
EP1 Consumer Edge Gateway	The first firm-controlled hop for any third-party AI traffic (API gateway tier)	Consumer and agent identity, principal scope, request and response envelope, rate and sequence controls, disclosure delivery	All surfaces	Route through the gateway; no code change
EP2 Agent Runtime	The orchestration layer that executes plans, steps and approvals for firm-hosted assistants and agents	Interaction and step lifecycle, context assembly, plan revisions, human-in-the-loop decisions, final answer assembly	Hosted assistants, backend agents, batch AI	Build on the platform runtime, or wrap a custom runtime with the SDK
EP3 Model Gateway	The only egress to any model provider, hosted or on-premises	Prompt and response capture, model identity and parameters, guardrail verdicts, token and cost accounting, vendor data-handling flags	Any model call in scope	Point the model client at the gateway; API-compatible with major providers

Enforcement point	Where it sits	What it owns	Mandatory for	How a team adopts it
EP4 Tool / MCP Gateway	The only path from an agent or an external MCP client to a firm tool or data service	Policy decision per call, full tool request and result, provenance stamping of every returned value, action-class enforcement	Every tool and MCP server	Register the tool; the gateway wraps it
Systems of record (supporting)	Data services behind EP4	As-of time, record identifier and content hash on every response	Data services that serve AI	Adopt the provenance response header standard

The AI Audit Plane is a platform service with six stages: ingest and validate, classify and redact, hash-chain and sign, store (WORM archive plus hot index), anchor externally, and verify and export. It is operated by a team that is organisationally separate from the teams producing events and from the teams investigating them.

The canonical audit record

Every enforcement point emits the same envelope, `AIAuditEvent`, so an investigator can join records from any layer on one set of identifiers without knowing which team built the workload. The envelope is versioned, the event types are closed, and unknown fields are rejected at ingest.

Correlation spine. Four identifiers travel on every event and are propagated in headers (W3C `traceparent` plus an `ai-interaction-id` baggage entry) across every hop, including into third-party MCP calls and back.

Identifier	Scope	Set by
`interaction_id`	One end-to-end request from a consumer to a final answer or completed workflow	EP1 on entry; a consumer may supply its own, which is recorded as `consumer_ref` but never used as ours
`trace_id` / `span_id`	The OpenTelemetry trace and span the event belongs to	Each enforcement point
`step_seq`	Monotonic step number within the interaction (plan revision, model call, tool call, approval)	EP2, or EP4 for direct MCP calls
`stream_id`	The hash chain the event is appended to (one per consumer tenant per day)	Audit Plane at ingest

Identity chain. Recorded as a list of verified links, each with the credential type and verification result, so a record never relies on a self-asserted claim.

Link	Example	Verified by
`consumer_org`	Advisory firm tenant id	mTLS client certificate or OAuth client, EP1
`consumer_system`	The firm's agent or application, with a signed agent attestation where supplied (agent name, version, model vendor they use)	EP1
`operator`	The human advisor driving the session, where the consumer passes an identity token	Federated identity, EP1
`principal`	The investor or account the request is for	Entitlement service at EP4, per call
`firm_agent`	Our agent or assistant definition and version	EP2
`firm_service`	The workload identity of the service making the call	SPIFFE identity at EP3 and EP4

Event types and required content. Payload fields are stored in full and content-addressed; the table shows what each layer must capture beyond the spine and identity chain.

Event type	Emitted by	Required content
`interaction.started` / `interaction.completed`	EP1	Surface, channel, consumer request envelope hash, final response hash, outcome (answered, refused, escalated, errored), disclosures delivered, total steps, total tokens, cost
`plan.created` / `plan.revised`	EP2	The agent's plan text, the model span that produced it, the tools it intends to call

Event type	Emitted by	Required content
`context.assembled`	EP2	Ordered list of every item placed in the context window: system prompt hash, tool manifest hash, prior turns by hash, retrieved chunks by provenance stamp, consumer-supplied content by hash with trust class
`model.called`	EP3	Provider, model id and version, region, data-handling contract id, zero-retention flag, full request body, full response body, decoding parameters, token counts, latency, guardrail input and output verdicts, finish reason
`tool.called`	EP4	Tool name and version, action class (read, derive, propose, execute), full arguments, policy decision and the rules that fired, principal scope checked, full result, result row and field counts, provenance stamps on every value, duration
`approval.requested` / `approval.decided`	EP2	What the approver was shown (rendered content hash), approver identity, decision, time-to-decide
`guardrail.triggered`	EP1, EP3, EP4	Classifier, verdict, confidence, action taken (block, redact, warn, allow)
`response.rendered`	EP1	Exact content delivered to the consumer, including citations, confidence markers and disclaimers, as rendered
`audit.gap`	Any	Emitted when a record could not be produced or delivered, with cause; the presence of a gap is itself audited

Minimum fields on every event: `event_id` (UUIDv7), `event_type`, `schema_version`, `occurred_at` (RFC 3339, UTC, microsecond), `emitter` (enforcement point and instance), the spine, the identity chain, `data_classification` (highest class present in the payload), `payload_hash` (SHA-256 of the canonical JSON payload), `prev_hash` (set at ingest), and `redaction_manifest` (which fields were tokenised and under which key id).

Payloads over 256 KB are written to content-addressed blob storage under the same retention lock and referenced by hash; the event keeps the first 4 KB inline for search.

Provenance and claim-to-source grounding

A statement the AI makes to a third party is defensible only if the record shows which source value it came from, that the value was what the system of record held at that moment, and that the model actually had that value in its context. The design achieves this with a provenance stamp on every value entering the model, a grounding map on every value leaving it, and a verifier that checks the two against each other.

Provenance stamp (inbound). EP4 attaches a stamp to every scalar, row or document chunk returned by a tool before it is placed in the context window. Data services supply the first four fields through a standard response header; EP4 computes the rest.

Field	Meaning
`source_system`	System of record or service name and environment
`record_ref`	Stable identifier of the record, row or document (account id, document id, position key)
`as_of`	The time the value was true in the source, not the time it was fetched
`content_hash`	SHA-256 of the canonical value
`tool_span`	The `tool.called` event that fetched it
`trust_class`	`firm_record`, `firm_derived`, `consumer_supplied`, `public_web`, `model_generated`
`access_basis`	The entitlement rule that permitted the principal to see it

Stamps are carried in-band as structured metadata beside the content (never embedded in the prose the model reads) and recorded in `context.assembled`. Consumer-supplied content and anything fetched from outside the firm is stamped `consumer_supplied` or `public_web` and passed through the injection classifier before the model sees it; the verdict is on the stamp.

Grounding map (outbound). The model is instructed, through the platform system prompt and a structured output contract, to emit citations as references to stamp ids rather than free text. EP2 then builds a grounding map for the final response: each sentence or figure is linked to zero or more stamps. EP2 records three measures with the `interaction.completed` event: the share of factual spans with at least one stamp, the share of stamps actually cited, and the list of unsupported factual spans. Where the surface is a hosted assistant, citations are rendered to the user and the rendered form is in `response.rendered`.

Verification. A grounding verifier runs out of band on a sample of interactions and on every interaction flagged by risk rules (transactional, high-value principal, complaint). It re-fetches the cited records as of the stamped time, recomputes the hash, and checks that the cited value supports the claim using a separate judge model whose own calls are logged under the same interaction. Mismatches raise a grounding incident to the owning team and to Compliance.

What this lets the firm say. For any statement delivered to an advisory firm the record answers: which record, in which system, as of when, with what hash, fetched by which tool under which entitlement, placed in which context, cited in which response, rendered how. For any statement without a stamp the record shows that it was model-generated and whether the surface labelled it as such.

Direct MCP consumers. When the advisory firm's own agent calls our MCP server we cannot see their prompt or their final answer. We still stamp every value we return and sign the tool result, so the firm can prove what we supplied, and the consumer can prove what they received. The MCP tool contract requires consumers to pass their `interaction_id` and operator identity on each call and the platform records both as consumer references.

Integrity and tamper evidence

The audit record must survive a motivated insider with production access, a compromised producing service, and a dispute in which the firm's own word is not enough. The design layers five controls so that altering, inserting or deleting a record after the fact is

detectable by a party outside the producing team, and provable to a party outside the firm.

1. **Content addressing at the source.** Every enforcement point computes `payload_hash` over the canonical JSON (RFC 8785 JSON Canonicalization Scheme) before transmission and signs the event with its workload identity key (SPIFFE SVID, short-lived, issued by the platform CA). A record whose producer signature does not verify is quarantined, not stored.
2. **Per-stream hash chain.** At ingest the Audit Plane assigns each event to a stream (one per consumer tenant per UTC day) and sets `prev_hash` to the previous event's `entry_hash`, where `entry_hash = SHA-256(prev_hash || payload_hash || event_id || occurred_at)`. A gap, reorder or edit breaks every subsequent link.
3. **Merkle batching and platform signature.** Every 60 seconds, or every 10,000 events, the stream's new entries are batched into a Merkle tree. The root is signed by the Audit Plane's HSM-held key and written as a checkpoint record into the stream itself. Any single event can be proven to belong to a signed checkpoint with a log-sized inclusion proof.
4. **External anchoring.** Each checkpoint root is submitted to an RFC 3161 timestamp authority operated by an independent provider, and the hourly roll-up of checkpoint roots is additionally published to a second independent anchor (a second TSA from a different vendor, or a transparency log in the style of Certificate Transparency or SCITT). A tampered checkpoint cannot be reconciled with the anchored root; a backdated one cannot obtain a matching timestamp.
5. **Write-once storage with retention lock.** The archive tier is object storage in compliance mode (S3 Object Lock or equivalent) with a legal-hold capability and the retention period set at write. No role, including the account root, can shorten or delete within the lock. The hot index is a derived copy; it is rebuilt from the archive, never the other way round.

Segregation of duties. Producers (application and platform teams) can only append through the ingest API. Custodians (the Audit Plane team) operate storage but hold no signing keys and no retention-override rights. Verifiers (Internal Audit, Compliance Technology) hold read access and run the continuous verification job. Key custody for the HSM signing key is split between Security and Internal Audit under dual control.

Continuous verification. A job owned by Internal Audit walks every stream daily, recomputes the chain, checks each checkpoint signature and its anchor, and compares the archive object count with the ingest counters. Any failure is a Severity 1 security

incident. The verification result is itself written as a signed record so the absence of a daily verification is detectable.

Export with proof. An export for an examiner, a court or an advisory firm carries the events, the inclusion proofs, the checkpoint signatures, the public keys and the TSA tokens, so the recipient can verify the records offline without trusting the firm's systems. The export tool is the only supported path out of the archive and every export is itself an audited event.

Storage, retention and access

Retention follows the firm's books-and-records schedule, not an engineering convenience: an AI interaction with an advisory firm is a business communication and, where it informs a recommendation or transaction, a record of that transaction. The default is 7 years in write-once storage, with the first 2 years readily accessible, which matches the firm's existing posture under SEC Rule 17a-4 and FINRA Rule 4511 and leaves Legal to shorten classes, never lengthen them under pressure.

Tier	Holds	Retention	Access
Hot index	Redacted events, searchable by every spine and identity field, full-text over prompts and responses	2 years, then dropped and rebuilt on demand from archive	Owning team (own workloads), Compliance, Security, Internal Audit
WORM archive	Every event and referenced blob in canonical form, signed and chained, compliance-mode lock	Per class below; lock set at write	Audit Plane custodians for operations; read through the export tool only
Restricted vault	Reversible-tokenisation keys and the raw form of fields redacted at ingest	Same as the archive class of the events they unlock	Dual control: Compliance plus Security, each access logged and reviewed monthly
Operational traces	OpenTelemetry spans and metrics sharing the correlation spine	30 days	Engineering

Retention classes. Each `interaction.completed` event is assigned a class by rules on surface, action class and outcome, and the class is written to every event in the interaction before the archive lock is set.

Class	Examples	Retention
Transactional	Any interaction with an `execute` or `propose` tool call: order proposals, account changes, document generation sent to a client	7 years, first 2 readily accessible; longer where the underlying record class requires it (account opening documents, for the life of the account plus 6 years)
Advisory communication	Answers about holdings, performance, fees, suitability or products delivered to an advisory firm	7 years

Class	Examples	Retention
Informational	Read-only answers with no principal data and no recommendation (product documentation, how-to)	3 years
Refused or blocked	Interactions stopped by policy or guardrail	7 years; these are the evidence that controls worked
Audit gap	`audit.gap` events and verification results	10 years

Legal hold. A hold is applied by stream, consumer, principal or interaction and overrides expiry. Hold application and release are dual-control actions by Legal and are themselves audited.

Residency. Archive buckets are in the jurisdiction of the consumer's contracting entity. Cross-border investigators access through the export tool, which applies the residency policy and records the transfer.

Service levels. Any interaction is retrievable by the spine identifiers within 15 minutes of occurrence. A regulator request scoped by consumer, principal and date range is fulfilled, with proofs, within 24 hours for the hot tier and 5 business days from archive. Ingest latency to a durable, chained write is under 5 seconds at the 99th percentile; events are not acknowledged to the producer before that write.

Cost control. Full content capture is non-negotiable for the decision path; cost is managed by content addressing (identical system prompts, tool manifests and documents are stored once), by compression at the archive tier, and by the retention classes above rather than by sampling.

Privacy, redaction and sensitive data in the logs

The audit record is the most concentrated store of client data the firm will hold, because it contains prompts, retrieved records and answers side by side. It is protected as a Restricted system, and sensitive fields are tokenised at ingest so that routine investigation works on tokens while the original value remains recoverable under dual control for the cases that need it.

Classification at ingest. The Audit Plane runs a classifier over every payload and tags the event with the highest class found. The classifier combines deterministic detectors (account numbers, tax identifiers, card numbers, dates of birth, the firm's own identifier formats) with a model-based detector for free-text personal data and material non-public information. The classifier's own verdict and version are on the event, so a missed detection can be found later by re-running the newer classifier over the archive index.

Tokenisation, not deletion. Fields in the regulated classes are replaced in the hot index and in the default export with a format-preserving token under a key held in the restricted vault. The archive keeps the token and the encrypted original side by side. The investigator sees that account `ACCT-7f3a…` was retrieved and that a figure was quoted; Compliance with a second approver can reveal the value. Nothing in the decision path is dropped, so the chain and the grounding map stay verifiable over tokens.

Data class	Treatment in hot index and default export	Reveal
Client identifiers (account, tax id, national id)	Format-preserving token	Compliance + Security dual control
Contact details and free-text personal data	Token with class label	Compliance + Security dual control
Financial values tied to a principal (balances, positions, performance)	Kept in clear within the owning team's and Compliance's scope; tokenised for cross-team analytics	Scope-based, logged
Material non-public information	Token; event flagged `mnpi` and restricted to Compliance and the information barrier owner	Information barrier owner
Credentials, keys and session tokens accidentally present in a prompt	Irreversibly redacted and a security event raised	Never
Model and prompt internals (system prompts, tool manifests)	Stored by hash with the content in the archive; visible to the owning team and Internal Audit	Owning team

Consumer-supplied content. Documents and messages an advisory firm sends into the system belong to a different controller. They are stored under the same lock, tagged with the consumer organisation, and included in the consumer's own export on request. Contracts with advisory firms state that their requests and our responses are retained for the regulatory period and may be produced to regulators.

Data subject rights. Erasure requests cannot delete records inside the retention lock. The platform handles them by revoking the tokenisation key material for that subject's fields once the regulatory basis for retention has expired, and by recording the request against the affected streams so that exports exclude the subject where the law allows.

Model provider processing. EP3 records, per call, the provider, region, contract id and whether zero-retention terms applied, so the firm can show which prompts a provider ever held and under what terms. Providers without contractual zero-retention or equivalent terms are not reachable from EP3 for in-scope workloads.

Access logging. Every read of the audit record, including by the investigation tools, is itself an `AIAuditEvent` in a dedicated stream with the reader's identity, the query, and the number of records and fields returned. Access patterns are reviewed monthly by Internal Audit.

Standards and regulatory mapping

The design reuses existing standards wherever one exists so that tooling, auditors and vendors already understand the artefacts; the firm-specific part is the `AIAuditEvent` envelope and the provenance stamp, both of which are thin layers over the standards below. Citations are to the public standards; the firm's regulatory interpretation is Compliance's, and this table is the engineering reading.

Standard or rule	What it gives us	Where it applies
OpenTelemetry GenAI semantic conventions	Attribute names for model, prompt, completion, token usage and tool-call spans; vendor instrumentation already emits them	EP2 and EP3 span attributes; the audit event carries the same names so traces and audit records join

Standard or rule	What it gives us	Where it applies
W3C PROV	A vocabulary for entity, activity and agent lineage	The provenance stamp and grounding map are expressible as PROV for export to external auditors
W3C Trace Context	`traceparent` and `tracestate` propagation across services and vendors	The correlation spine, including propagation to and from third-party MCP clients
Model Context Protocol specification	Tool, resource and prompt primitives, request and result shapes	EP4 logs the MCP messages in their native form; the MCP `_meta` field carries the spine to consumers
RFC 8785 JSON Canonicalization	Deterministic serialisation for hashing	`payload_hash` and `entry_hash`
RFC 3161 Time-Stamp Protocol	Independent proof that a hash existed at a time	Checkpoint anchoring
RFC 6962 Certificate Transparency and IETF SCITT	Merkle-tree transparency log design, inclusion and consistency proofs	Stream checkpoints and export proofs follow the same construction
NIST SP 800-92 log management	Baseline for log generation, protection, retention and analysis	Audit Plane operating procedures
NIST AI Risk Management Framework	Govern, map, measure, manage; expectations for traceability and documentation of AI systems	Governance section; the record is the evidence for Measure and Manage
ISO/IEC 42001 AI management system	Auditable AI management controls, including logging and impact assessment	Control mapping for certification; the conformance test is the control evidence
ISO/IEC 27001 Annex A logging and monitoring controls	Log protection, clock synchronisation, access to logs	Integrity section and access logging

Standard or rule	What it gives us	Where it applies
EU AI Act record-keeping for high-risk systems	Automatic logging over the system lifetime, traceability of operation, retention appropriate to purpose	Applies if any surface is classified high-risk for EU principals; the design already exceeds the logging duty
SEC Rule 17a-4 and FINRA Rule 4511	Books and records retention; write-once or audit-trail electronic storage; 6-year baseline, first 2 years accessible; records retained by a third party need an access undertaking	Retention classes, WORM tier, export tool, vendor contracts where a provider holds any record
FINRA Regulatory Notice 24-09	FINRA's reminder that existing rules (supervision, communications, recordkeeping) apply to generative AI	The rationale for treating AI output to advisory firms as a supervised communication
SEC Regulation S-P and GLBA	Safeguarding and disposal of client information; breach notification	Privacy section; access logging and vault controls
OWASP Top 10 for LLM Applications and MITRE ATLAS	Threat catalogue for prompt injection, data leakage, excessive agency, insecure tool use	Risk model and the guardrail events; SIEM detections are written against these
CloudEvents	A transport-neutral event envelope	The `AIAuditEvent` is carried as a CloudEvent on the ingest bus

The 7-year default in the retention section is the firm's existing policy choice above the regulatory baseline and should be confirmed by Compliance for each surface; the engineering design treats the class, not the number, as the fixed point.

Platform enablement: the paved road

A team should get compliant audit logging by choosing the platform defaults, not by reading this document. The platform offers four integration modes, ordered from least to most effort, and a conformance test that certifies the result; a workload that passes conformance is cleared for third-party exposure without a bespoke review of its logging.

Mode	For whom	What the team does	What the platform does
Managed runtime	Teams building assistants or agents on the platform agent runtime	Define the agent (prompts, tools, policies) in the runtime's manifest	Everything: EP1 through EP4 are built in; the team never sees the audit API
Gateway only	Teams with an existing application that calls models and tools	Point model clients at EP3 and register tools with EP4; route external traffic through EP1	Captures model and tool events and the envelope; the team adds step and approval events through the SDK if it runs its own loop
SDK	Teams with a custom orchestration loop in Python, TypeScript, Java or .NET	Wrap the loop with the SDK's `interaction` and `step` context managers; use the SDK's model and tool clients	Emits the full event set, propagates the spine, buffers and retries, enforces fail-closed on `execute` tools
Sidecar	Teams that cannot change code (vendor products, legacy)	Deploy the audit sidecar as an egress proxy for model and tool traffic	Captures model and tool events from the wire; cannot capture plans or approvals, so the workload is limited to the read-only tier

What the SDK does for a team.

- Opens an interaction and propagates the spine on every outbound call, including MCP calls to and from third parties.

- Wraps model and tool clients so prompts, responses, arguments and results are captured without the team serialising anything.
- Attaches provenance stamps from data-service response headers automatically, and warns at build time when a tool returns values without stamps.
- Applies the fail-closed rule: an `execute` or `propose` tool call does not proceed until the preceding events are durably acknowledged.
- Buffers to local encrypted disk on ingest outage, emits `audit.gap` with the cause, and replays in order when ingest returns.
- Ships a local developer mode that writes events to a file and renders them in the same investigation UI, so a developer sees the audit trail of their own test run before a code review.

Conformance. A workload is certified by a test harness that drives it through a fixed scenario set: a grounded answer, an unsupported answer, an over-scoped tool call, an injected document, a transactional action with and without approval, and an ingest outage. The harness checks that every expected event arrived, that the chain verifies, that stamps are present on every value, that the grounding map matches the rendered response, and that the fail-closed rule held. Certification is recorded against the workload identity and re-run on every production release.

Tool registration. A tool or MCP server enters EP4 through a registry entry: owner, action class, input and output schemas, data classification of outputs, the entitlement rule it enforces, and whether it returns provenance stamps. The registry is the tool manifest hash that appears on every `context.assembled` event, so an investigator can see exactly which tool versions were available to the model.

Investigation tooling. One UI, owned by the Audit Plane team, replays an interaction as a timeline: the context the model saw at each step, the plan, each tool call with its policy decision, each response with its grounding map, and the chain proof. The same UI serves engineers debugging in non-production, Compliance reviewing a complaint and Internal Audit sampling controls, with field visibility set by role.

Developer experience commitments. A new managed-runtime agent produces a complete audit trail in its first deployment with zero audit-specific code. Gateway and SDK adoption for an existing workload is budgeted at under two engineer-weeks, including conformance. The platform team publishes a reference implementation and a sample agent that passes conformance, and runs office hours for the first two quarters.

Governance, control gates and maturity tiers

The standard is enforced at three gates in the existing delivery lifecycle rather than by a new committee, and a workload's permitted exposure is set by the tier it has demonstrated, not by the tier it intends to reach.

Control gates.

1. **Design gate (architecture review).** The team names the surface, the integration mode, the tools and their action classes, the data classes in play and the retention class. Review confirms the four enforcement points are on the path and that every data service in scope returns provenance stamps or has a dated plan to.
2. **Release gate (CI/CD).** The conformance harness runs on every release candidate; a failing result blocks deployment to any environment reachable by a third party. Tool registry entries and system prompt hashes are versioned with the release.
3. **Exposure gate (Compliance and Security sign-off).** Before a consumer is onboarded to a surface, Compliance confirms the retention class and disclosures, Security confirms the consumer's identity and attestation setup, and the Audit Plane team confirms the stream is anchored and the verification job covers it.

Maturity tiers. A workload's tier is recorded in the registry and determines which action classes EP4 will allow it to call for a third party.

Tier	Evidence required	Permitted for third parties
Tier 1 Observed	Model and tool events captured (gateway or sidecar), chain verifies, retention class set	Read-only answers over non-principal data
Tier 2 Grounded	Tier 1 plus provenance stamps on every value, grounding map on every response, context assembly logged	Read-only answers over principal data, with citations rendered
Tier 3 Accountable	Tier 2 plus plan and approval events, fail-closed on `execute`, conformance passed including injection and outage scenarios	`propose` actions (draft orders, document packs for human review)

Tier	Evidence required	Permitted for third parties
Tier 4 Transactional	Tier 3 plus human approval policy per action class, grounding verifier on 100% of transactional interactions, examiner export exercised	`execute` actions on the principal's behalf

Roles.

- Enterprise Architecture owns the standard, the schema and the tier definitions, and reviews changes quarterly.
- The AI Platform team owns the enforcement points, SDK, sidecar, conformance harness and registry.
- The Audit Plane team, reporting outside the AI Platform line, owns ingest, storage, anchoring, export and the investigation UI.
- Compliance owns retention classes, disclosure requirements and the exposure gate; Legal owns holds and residency.
- Security owns consumer identity, attestation, key custody with Internal Audit, and SIEM detections.
- Internal Audit owns continuous chain verification and the monthly access review, and samples grounding incidents.
- Each workload has a named accountable owner who receives grounding incidents and audit gaps for their workload.

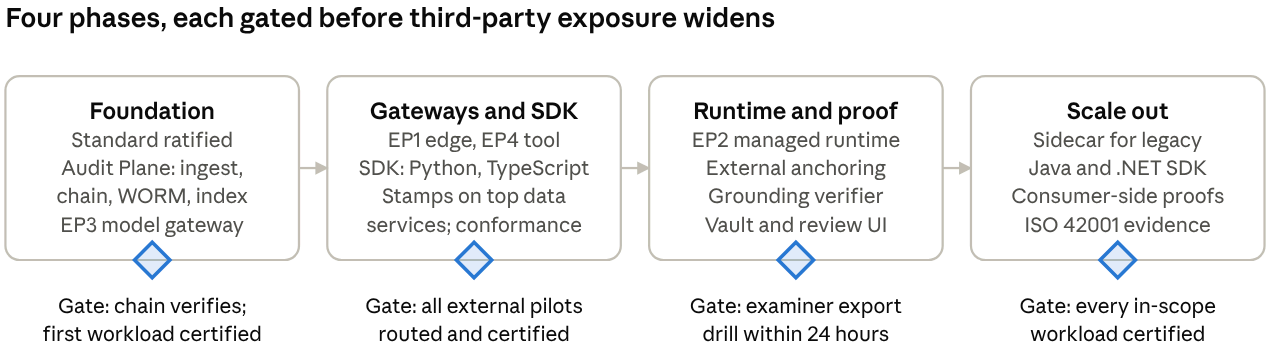
Metrics reported monthly to the AI governance forum. Share of in-scope workloads at each tier; audit gap events per million steps; grounding coverage and unsupported-claim rate per surface; verification job pass rate; time to fulfil the last regulator or consumer export; count of reveal operations from the vault and their approvers.

Exceptions. An exception to any requirement is time-boxed to 90 days, approved by Enterprise Architecture and Compliance, recorded in the registry so that it appears on every event the workload emits, and reported in the monthly metrics until closed.

Roadmap and open decisions

Delivery runs in four phases, and the gate at the end of each is a demonstrated capability, not a date: the firm widens third-party exposure only as far as the evidence supports. Phase order is fixed because each phase's enforcement point is a prerequisite for the

next tier of permitted actions.



Adoption roadmap · 4 phases, 4 gates

The model gateway comes first because it is the cheapest single control with the widest coverage; the managed runtime comes third because plans and approvals are only needed once teams are allowed to propose or execute actions.

Phase sequencing in detail.

1. **Foundation.** Ratify this standard and the `AIAuditEvent` schema. Stand up the Audit Plane with ingest, per-stream chaining, the WORM archive and the hot index. Put EP3 in front of every model provider in use and cut over existing pilots. Exit when the daily verification job passes for 30 consecutive days and one workload is certified at Tier 1.
2. **Gateways and SDK.** Deliver EP1 and EP4, the Python and TypeScript SDKs and the conformance harness. Add provenance response headers to the data services that back the current external pilots. Exit when every externally exposed pilot is routed through all applicable enforcement points and certified at Tier 2.
3. **Runtime and proof.** Deliver the managed agent runtime with EP2, external anchoring to two independent anchors, the grounding verifier, the tokenisation vault and the investigation UI. Exit when a simulated examiner request is fulfilled with proofs within 24 hours and one workload is certified at Tier 3.
4. **Scale out.** Deliver the sidecar, the Java and .NET SDKs, consumer-side proof verification for MCP clients, and the ISO/IEC 42001 control evidence pack. Exit when every in-scope workload is certified and no exception is older than 90 days.

Open decisions for the governance forum.

- [ ] Confirm the 7-year default and the retention class rules with Compliance for each surface, and whether advisory communications over the hosted assistant fall under existing communications-supervision review sampling.

- [ ] Choose the two independent anchors (timestamp authority vendors, or one TSA plus a transparency log) and who holds the relationship.
- [ ] Decide whether consumer-supplied content is retained in full or by hash plus a consumer-held copy, per contract.
- [ ] Agree the fail-open allowance for read-only answers during an ingest outage: the maximum buffered duration before the surface degrades to a static response.
- [ ] Set the grounding verifier sampling rate for non-transactional surfaces and the threshold at which an unsupported-claim rate triggers a surface pause.
- [ ] Name the accountable owner for the Audit Plane outside the AI Platform reporting line.
- [ ] Decide whether the firm publishes its checkpoint roots to advisory firms so they can verify tool results they received, and in which phase.